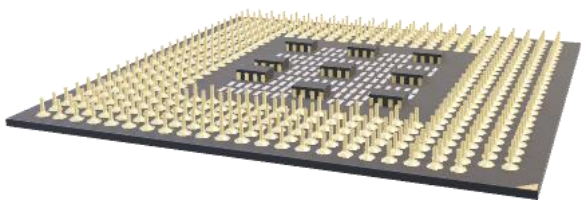
Universitatea Tehnica din Cluj-Napoca
Catedra de Calculatoare

# Simularea unei Scene Urbane Interactive cu Iluminare Dinamica si Randare Fotorealista

*Student : Turceniuc Mihai-Cezar*
*Grupa : 30235*
*Indrumator : Sabou Adrian*
*Data : 16.01.2026*



# Table of Contents

1. Prezentarea temei ..... 3
2. Scenariul ..... 3
2.1. Descrierea scenei si a obiectelor ..... 3
2.2. Functionalitati ..... 4
3. Detalii de implementare ..... 4
3.1. Functii si algoritmi ..... 4
3.1.1. Generarea Umbrelor (Shadow Mapping & PCF) ..... 4
3.1.2. Detectia Coliziunilor (OBB - Oriented Bounding Box) ..... 6
3.1.3. Camera Third-Person (Interpolare) ..... 7
3.1.4. Iluminare Multipla (Spotlights & Optimizare) ..... 8
3.2. Modelul grafic ..... 9
3.3. Structuri de date ..... 9
3.4. Ierarhia de clase ..... 9
4. Prezentarea interfetei grafice utilizator / Manual de utilizare ..... 10
5. Concluzii si dezvoltari ulterioare ..... 11
6. Referinte ..... 11

# 1. Prezentarea temei

Proiectul presupune dezvoltarea unei aplicatii 3D interactive care simuleaza un mediu urban, punand accent pe realismul vizual si pe interactiunea utilizatorului cu obiectele scenei. Aplicatia este construita folosind limbajul C++ si API-ul OpenGL , utilizand pipeline-ul programabil.

Obiectivele principale au fost:

- Implementarea unui model de iluminare complex (Blinn-Phong) care suporta multiple tipuri de surse de lumina (directionala si de tip spot).
- Generarea umbrelor dinamice de inalta rezolutie.
- Simularea fizicii de baza prin detectia coliziunilor intre vehicule si elementele statice ale orasului.
- Crearea unei experiente captivante prin efecte sonore si control fluid al camerei.

Biblioteci externe utilizate: GLFW (fereastra/input), GLM (matematica), irrKlang (audio), stb_image (incarcare texturi).

# 2. Scenariul

## 2.1. Descrierea scenei si a obiectelor

Scena reprezinta un cartier urban detaliat, compus din:

- Elemente statice: Cadrul natural specific iernii, cladiri (blocuri), vegetatie (copaci), stalpi de iluminat si un finish line.
- Elemente dinamice (Vehicule): Trei modele de masini detaliate (Bmw M3, Porsche 911, masina de politie).

Modelarea obiectelor: Pentru a permite animatia realista, masinile nu sunt incarcate ca un singur obiect, ci sunt descompuse in sub-obiecte: Car Body (sasiu) si cele 4 roti (sf, ss, df, ds). Acest lucru permite rotirea individuala a fiecarei roti in functie de viteza de deplasare, independent de translatia sasiului.

## 2.2. Functionalitati

Utilizatorul are control total asupra simularii:

1. Selectarea Vehiculului: Poate comuta instantaneu intre cele trei masini disponibile.
2. Driving Simulator: Controlul masinii (accelerare, virare) cu feedback vizual (rotatia rotilor) si audio (melodie specifica / sirene).
3. Iluminare Interactiva:

- Ciclul zi-noapte prin controlul pozitiei soarelui.
- Aprinderea/stingerea automata a farurilor (Spotlights).
- Girofaruri functionale pentru politie cu lumini alternante.

4. Sistem de Camere:

- Free Cam: Explorare libera.
- NFS Style: Camera care urmareste spatele masinii cu o miscare dinamica si fluida.
- Tour Mode: O prezentare cinematica predefinita a scenei de obiecte.

# 3. Detalii de implementare

## 3.1. Functii si algoritmi

### 3.1.1. Generarea Umbrelor (Shadow Mapping & PCF)

Solutie: Am utilizat tehnica Shadow Mapping facand 2 trasari ale scenei:

1. Pasul 1 (Depth Pass): Scena este randata din perspectiva sursei de lumina (Soare) intr-un Framebuffer (FBO) care stocheaza doar adancimea (Depth Map).
2. Pasul 2 (Render Pass): Scena este randata normal, iar pentru fiecare pixel se verifica daca este in umbra comparand adancimea sa cu cea din Depth Map.

Implementare iluminare: am folosit o proiectie ortogonala deoarece soarele este o sursa directionala (raze paralele):

```
glm::mat4 computeLightSpaceTrMatrix() {
   // pozitionam camera la pozitia soarelui uitandu se catre (0,0,0)
   glm::vec3 lightPos = glm::normalize(lightDir) * 75.0f;  // inmultim cu 75 pt distanta aproximativa
   glm::mat4 lightView = glm::lookAt(lightPos, glm::vec3(0.0f), glm::vec3(0.0f, 1.0f, 0.0f));
   GLfloat xSize = 150.0f; // latimea zonei cu umbre
   GLfloat ySize = 150.0f; // lungimea zonei cu umbre
   GLfloat zNear = 0.1f;
   GLfloat zFar = 300.0f;  // adancimea
   // proiectia ortogonala a razelor de soare
   glm::mat4 lightProjection = glm::ortho(-xSize, xSize, -ySize, ySize, zNear, zFar);
   glm::mat4 lightSpaceTrMatrix = lightProjection * lightView;
   return lightSpaceTrMatrix;
}
```

Implementare Shader (PCF - Percentage Closer Filtering): Pentru a evita marginile pixelate ale umbrelor (aliasing), s-a implementat PCF, care face media valorilor de umbra din jurul pixelului curent (o matrice 5x5).

Basic.frag – implementare umbre:

```
float computeShadow(){
   // perform perspective divide
   vec3 normalizedCoords = fragPosLightSpace.xyz / fragPosLightSpace.w;
   // Transform to [0,1] range
   normalizedCoords = normalizedCoords * 0.5 + 0.5;
   if (normalizedCoords.z > 1.0f)
      return 0.0f;
   // Get closest depth value from light's perspective
   float closestDepth = texture(shadowMap, normalizedCoords.xy).r;

   // Get depth of current fragment from light's perspective
   float currentDepth = normalizedCoords.z;

   // Check whether current frag pos is in shadow
   float bias = max(0.005f * (1.0f - dot(normalize(fNormal), normalize(lightDir))), 0.001f);
   float shadow = 0.0f;
   float texelSize = 1.0f / 8192.0f;
```

```
    // Percentage Closer Filtering => verificam mai multi pixeli din jurul umbrei
    // nu doar un pixel si facem media pentru umbre mai smooth
    for(int i=-2; i<=2; i++){
        for(int j=-2; j<=2; j++){   // facem media a 9 pixeli [-1,1]
            vec2 offset = vec2(i, j) * texelSize;   // offset pt fiecare vecin
            float pcfDepth = texture(shadowMap, normalizedCoords.xy + offset).r;    // citim depth ul din
shadowMap
            shadow += currentDepth - bias > pcfDepth ? 1.0f : 0.0f;
        }
    }
    shadow /= 25.0f;
    return shadow;
}
```

### 3.1.2. Detectia Coliziunilor (OBB - Oriented Bounding Box)

Motivare: O simpla verificare sferica sau AABB (Axis Aligned Bounding Box) ar fi fost imprecisa deoarece masinile si cladirile se rotesc, iar "cutia" de coliziune nu ar mai fi aliniata cu axele lumii. Solutie: Am implementat algoritmul OBB. Se transforma pozitia masinii in sistemul de coordonate local al cladirii (luand in calcul rotatia cladirii), simplificand problema la o verificare de tip "punct in dreptunghi".

Main.c - functia de verificare a coliziunilor:

```
bool checkBuildingCollision(glm::vec3 carPos, float carRadius, const OrientedBuildingBox& b) {
    // vectorul distanta de la pozitia cladirii la masina
    glm::vec3 delta = carPos - b.position;

    // rotatie inversa a masinii pt a fi aliniata catre cladire
    float angleRad = glm::radians(-b.rotation);
    float c = cos(angleRad);
    float s = sin(angleRad);

    float localX = delta.x * c + delta.z * s;          // BB min si max
    float localZ = -delta.x * s + delta.z * c;

    // cautam cel mai apropiat punct de pe marginea bounding box ului
    float padding = 0.2f;
```

```
    float halfX = (b.size.x / 2.0f) + padding;
    float halfZ = (b.size.z / 2.0f) + padding;

    float closestX = glm::clamp(localX, -halfX, halfX);
    float closestZ = glm::clamp(localZ, -halfZ, halfZ);

    // distante de la masina si cel mai apropiat punct
    float distX = localX - closestX;
    float distZ = localZ - closestZ;

    return (distX * distX + distZ * distZ) < (carRadius * carRadius);       // verificam distanta pt
coliziune
}
```

### 3.1.3. Camera Third-Person (Interpolare)

Pentru a crea efectul de camera dinamica (specific jocurilor de curse precum Need For Speed), camera nu este fixata rigid de masina, ci is actualizeaza pozitia curenta interpoland intre pozitia initiala si cea dorita.

Main.c – camera de tip “third-person”:

```
void updateNFSCamera() {

    // masina curenta folosita
    glm::vec3* currentCarPoz = getSelectedCarPos();
    // directia masinii
    GLfloat* currentAngle = getSelectedCarAngle();
    float carAngleRad = glm::radians(*currentAngle);
    // vectorul forward pt directie
    glm::vec3 forwardDir = glm::normalize(glm::vec3(cos(carAngleRad), 0.0f, -sin(carAngleRad)));

    // pozitia camerei
    glm::vec3 targetCamPos = *currentCarPoz - (forwardDir * camOffsetDistance);
    targetCamPos.y += camOffsetHeight;

    // interpolare liniara intre pozitia curenta si cea "ideala" pt a da un efect de smooth placing pt
camera
    glm::vec3 currentPos = myCamera.getPosition();
    glm::vec3 smoothedPos = glm::mix(currentPos, targetCamPos, smoothFactor);
```

```
    // actualizam pozitia camerei
    myCamera.setPosition(smoothedPos);

    // punctul catre care priveste camera
    glm::vec3 lookAtPoint = *currentCarPoz + (forwardDir * 3.0f) + glm::vec3(0.0f, 1.0f, 0.0f);
    myCamera.setTarget(lookAtPoint);
}
```

### 3.1.4. Iluminare Multipla (Spotlights & Optimizare)

Am implementat 4 surse de lumina de tip spot (faruri stanga/dreapta, girofaruri). Optimizare: Pentru a mentine performanta, citirea texturilor in shader se face o singura data per pixel, nu in interiorul buclei for care calculeaza fiecare lumina.

Fragment Shader - basic.frag:

```
void main()
{   // calculam texturile o singura data la inceput
    vec4 colorTexture = texture(diffuseTexture, fTexCoords);
    vec3 diffuseColor = colorTexture.rgb;
    vec3 specularColor = texture(specularTexture, fTexCoords).rgb;

    // compute alpha testing for transparent background
    if(colorTexture.a < 0.1){
        discard;
    }
    // compute directional light (soarele)
    ...
    // compute shadow
    float shadow = computeShadow();
    // compute final vertex color
    vec3 finalColor = (ambient + (1.0 - shadow) * (diffuse + specular));
    // compute the 4 spot lights to finalColor
    for(int k = 0; k < 4; k++){
        finalColor += computeSpotLight(spotLight[k], normalEye, fPosition, viewDir, diffuseColor,
specularColor);
    }
    // compute fog
    float fogDistance = length(viewPos - fPosition); // distanta de la camera la texel
```

```
    float fogFactor = exp(-pow(fogDistance * fogDensity, 2));
    fogFactor = clamp(fogFactor, 0.0f, 1.0f);
    vec3 fogColor = vec3(0.5f, 0.5f, 0.5f); // culoare ceata
    vec3 colorWithFog = mix(fogColor, finalColor, fogFactor);   // combinam culorile
    fColor = vec4(colorWithFog, 1.0f);  // culoare finala
}
```

## 3.2. Modelul grafic

Aplicatia utilizeaza doua programe shader principale:

1. Depth Map Shader: folosit pentru generarea hartii de umbre, fara niciun output de culoare; acesta scrie automat adancimea in buffer pentru fiecare pixel, util in deciderea daca acesta este in umbra sau nu
2. Basic Shader: Realizeaza randarea finala. Implementeaza:
    - Modelul de iluminare: Blinn-Phong cu componentele: Ambient(lumina de fundal), Diffuse(culoarea de baza) si Specular(defineste shininess ul)
    - Calculul factorului de umbra.
    - Mixarea culorii finale cu culoarea de ceata (mix(fogColor, finalColor, fogFactor)), calculata exponential in functie de distanta.

## 3.3. Structuri de date

Structuri folosite:

- OrientedBuildingBox: Stocheaza pozitia (vec3), dimensiunile (vec3) si rotatia (float) unei cladiri pentru rezolvarea coliziunilor.
- CameraFrame: Folosita in vectorul sceneTour pentru a actualiza fiecare frame de prezentare cu pozitia si targetul actual al camerei.

Structuri lumini de tip spot:

- struct SpotLight: stocheaza proprietatile fizice ale luminii (pozitie, directie, cutOff, atenuare constanta, liniara si patratica).

## 3.4. Ierarhia de clase

Proiectul este modularizat pentru a separa logica:

- Clasa Camera: gestioneaza matricea View si vectorii Euler (Yaw, Pitch), returneaza si seteaza pozitia si target ul camerei pentru pozitionarea acesteia in scena de obiecte si schimbarea vizualizarii
- Clasa Shader(Vertex, Fragment, depthMap si Skybox): procesarea varfurilor si informatiilor acestora pentru calcularea umbrelor, culorii si pozitiei
- Clasa Model3D: utilizarea TinyObjLoader pentru a incarca fisierele de tip .obj si pentru generarea buffer-lor OpenGL (VAO, VBO si EBO).
- Clasa SkyBox: gestioneaza textura de tip Cubemap folosind gl_clamp pentru potrivirea si lipirea marginilor celor 6 fete

# 4. Prezentarea interfetei grafice utilizator / Manual de utilizare

Interactiunea se realizeaza prin tastatura si mouse, aceasta aplicandu-se direct asupra scenei 3D conform urmatorului tabel:

| Tasta | Actiune | Descriere |
|---|---|---|
| W, A, S, D | Deplasare | misca camera (mod liber) sau conduce masina |
| Mouse | Orientare | roteste privirea camerei |
| 1 | Camera mode | activeaza/dezactiveaza camera de tip “third-person” (NFS) |
| 2, 3, 4 | Selectare Masina | 2: BMW, 3: Porsche, 4: Politie |
| 5, 6, 7 | Mod vizualizare scena | 5: Solid, 6: Wireframe, 7: Poligonal |
| 8, 9 | Ceata | 8: scade densitatea cetii, 9: creste densitatea cetii |
| Sageti Sus, Jos, Stanga, Dreapta | Soare | modifica pozitia soarelui pe cer (modifica umbrele) |
| M / N | Cursor | M: activeaza cursorul, N: dezactiveaza cursorul |

# 5. Concluzii si dezvoltari ulterioare

Proiectul a reusit sa integreze cu succes elemente complexe de grafica computerizata intr-o aplicatie coerenta si performanta. Puncte forte:

- Implementarea corecta a Shadow Mapping la rezolutie inalta (4 * 2048 resolution) oferind umbre detaliate (folosind PCF pentru aproximarea mai buna a umbrelor).
- Sistemul de coliziuni OBB este functional si previne trecerea prin cladiri indiferent de unghiul de coliziune cu masinile
- Optimizarea shaderului calculand o singura data la inceput texturile iar apoi folosind aceste variabile in calculul iluminarilor si umbrelor

Dezvoltari ulterioare:

- Implementarea unui sistem de particule pentru a simula gazele de esapament sau ploaia
- Implementarea unei fizici realiste de control si viteza a masinilor
- Implementarea reflexilor realiste de pe suprafetele masinilor folosind Environment Mapping

# 6. Referinte


[1] **Real-Time Rendering, 4th edition:**
http://mutantstargoat.com/~nuclear/tmp/realtime_rendering_4ed.pdf

[2] **LearnOpenGL - Graphics Programming:**
https://learnopengl.com/book/book_pdf.pdf

[3] **Moduri de viualizare a scenei solid, wireframe si polygonal:**
https://registry.khronos.org/OpenGL-Refpages/gl4/html/glPolygonMode.xhtml

[4] **Libraria irrKlang pentru sunete:**
https://www.ambiera.com/irrklang/docu/index.html#links

[5] **Cubemap si Skybox - youtube tutorial:**
https://www.youtube.com/watch?v=8sVvxeKI9Pk

[6] **Camera placement in OpenGL - youtube tutorial:**
https://www.youtube.com/watch?v=lsOoo3Q4_ag

[7] **Tutoriale blender - playlist youtube:**
https://www.youtube.com/playlist?list=PLrgcDEgRZ_kndoWmRkAK4Y7ToJdOf-OSM